

UstaadAssist

A semester assistant for university teachers: what it does, why it exists, how to use every page, and the algorithms behind it.

Team guide — for testing the app and showing it to teachers
React Native (Expo) · Node.js + Express · PostgreSQL · Supabase
28 September 2026

Contents

1. What UstaadAssist is, and why it exists

The problem: teachers are the user nobody builds for
What actually goes wrong in a semester
What UstaadAssist does differently
The rule behind every screen: minimum input
How it helps the teacher, in one table

2. How the system fits together

What happens when the teacher signs in
What happens when the teacher uploads a file
Tools for testing
The words used in this guide

3. Using the app: signing in and setting up a course

3.1 Sign in, create account, forgot password
3.2 My courses
3.3 New course: step 1, course details
3.4 New course: step 2, your topics
3.5 New course: step 3, holidays
3.6 Plan generated
3.7 Clone a course (new semester from an old one)

4. Using the app: running the semester

4.1 Dashboard tab (the course home)
4.2 Plan tab: the teaching plan
4.3 A class's details: conducted or cancelled
4.4 Plan updated ("what moved")
4.5 Edit topics
4.6 Catch-up options (the deficit screen)
4.7 Students tab: adding the class list
4.8 Review class list (mandatory)
4.9 Student list and student profile
4.10 Attendance
4.11 Assessment tab
4.12 Material tab
4.13 Reports
4.14 Course settings

5. The algorithms: what they are, where they run, and why

5.1 Summary
5.2 Slot generation: which dates can have a class
5.3 Topic allocation: which topic on which date
5.4 Assessment validation: a quiz never comes before its topics
5.5 Replanning: freeze the past, rebuild the future
5.6 Deficit resolution: three ways to recover
5.7 Schedule health
5.8 Reading a class list photo: voting between readings
5.9 Results: one calculation for the screen and the PDF

6. Showing it to a teacher, and testing it

6.1 A ten-minute demonstration
6.2 Test checklist
6.3 Worth knowing while testing

1. What UstaadAssist is, and why it exists

A phone app that plans a university teacher's whole semester for one course, and keeps that plan correct as real life gets in the way.

The teacher sets a course up once: its name, the semester dates, the days it is taught, and the list of topics. From that, UstaadAssist **builds the teaching timetable itself**: which topic is taught on which date, week by week, skipping public holidays. When a class is cancelled or a topic takes longer than expected, the app **rebuilds the rest of the semester on its own** and shows the teacher exactly what moved.

Around that plan, the app handles the teacher's daily work: taking attendance, entering marks, calculating weighted totals and letter grades, keeping course material, and producing the three documents a teacher is asked for at the end of a term (a result sheet, an attendance report and a course delivery report).

The problem: teachers are the user nobody builds for

Look at the software a university already has. Student portals and learning platforms such as Moodle or Google Classroom serve **students**. ERP and admissions systems serve **the administration**. Timetabling serves **the department**, and result systems serve **the examination office**. Every one of these asks the teacher to **feed it** data. None of them helps the teacher **do the job**: decide what to teach on which day, notice when the course is falling behind, and recover from it before the final exam.

So in practice a teacher still runs the semester from a paper register, an Excel sheet for marks, a course outline written once in week one and never updated, and memory. That works until the semester stops going to plan, which it always does.

What actually goes wrong in a semester

- **Classes are lost.** A public holiday, a strike, a university event, the teacher is ill, a class is cancelled at short notice. Each lost class pushes every later topic back, but the written outline still shows the old dates.
- **Topics overrun.** "Normalization" was planned for two classes and needed three. The teacher now has to work out, by hand, what still fits.
- **Quizzes fall before their topics.** A quiz dated in week one of the plan is still on that date after the topics it covers have slipped two weeks. Students are tested on material they have not been taught.
- **Nobody notices the course is behind until it is too late.** By the time the teacher sees that three topics will not fit, the options for recovering are few.
- **End-of-term paperwork is assembled by hand.** Attendance percentages, the list of students below the attendance threshold, weighted totals, letter grades, and a summary of every class taught are copied between registers and spreadsheets, where mistakes creep in.

What UstaadAssist does differently

UstaadAssist is **not** a record-keeping app where the teacher types everything in and gets the same thing back. Its core is a **planner**: a set of algorithms that produce a plan, check it, and repair it. From semester dates, class days and topic titles it works out every real class date and what is taught on each. It keeps every quiz after the topics it covers. When a class is cancelled it rebuilds the rest of the semester and lists what moved. It notices when the course is behind, and when time runs out it offers three calculated ways to recover. From marks alone it produces weighted totals, grades and the result sheet.

The rule behind every screen: minimum input

A teacher will not type for forty minutes before seeing anything useful. So the app follows one rule everywhere:

Ask for the minimum, generate a draft, let the teacher correct it.

- Topics are typed as one block of text, **one topic per line**. There is no "add topic" form to repeat twenty times. A course outline can also be photographed or uploaded instead.
- Only the topic **title** is required. How many classes each topic needs, its minimum, and its priority all start at sensible defaults (1, 1, normal) and are corrected after the teacher has seen the plan.
- Students are **never typed**. The teacher photographs the printed class list, or uploads the PDF the department already sent, and the app reads roll numbers and names from it. The teacher then checks the result on a review screen before anything is saved.
- Public holidays are **already filled in**. The teacher only switches off the ones their university does not observe.
- Attendance starts with **everyone present**. The teacher taps only the students who are absent.
- Marks are entered on a **number keypad** that moves to the next student automatically.

How it helps the teacher, in one table

Task	Without UstaadAssist	With UstaadAssist
Planning the semester	Written by hand in week one, never updated	Generated in seconds from dates and topic titles
A class is cancelled	Re-plan every later date by hand, or not at all	Tap "Mark cancelled"; the rest of the semester is rebuilt
A topic overruns	Guess what still fits	Change "classes needed"; the plan and quiz dates follow
Quiz dates	Easy to leave a quiz before its topics	Never allowed: the quiz is moved and the reason shown
Falling behind	Noticed near the end	"You are 1 week behind your plan" on the home screen
Running out of time	Cut topics by instinct	Three calculated options, each saying how much it recovers
Class list	Typed student by student	Photographed or uploaded, then checked
Attendance	Paper register, percentages by hand	Tap absentees; percentages and the below-threshold list are automatic
Grades	Spreadsheet formulas	Weighted totals and letter grades from the teacher's own weightage and grade scale
End-of-term documents	Assembled by hand	Result sheet, attendance report and course delivery report as PDFs

2. How the system fits together

There are three pieces. It helps to know which piece is responsible when something goes wrong during testing.

Piece	What it does	Where it runs
The mobile app	Every screen the teacher sees. Signs the teacher in, uploads photos and files, and calls the backend for everything else.	The phone (built with React Native and Expo)
The backend	The planner, attendance, grading and reports. Checks who is calling and only ever returns that teacher's own data.	Vercel: https://ustaad-assist-backend.vercel.app
Supabase	Three services: sign-in (accounts and passwords), the PostgreSQL database, and file storage for photos and course material.	Supabase's servers

What happens when the teacher signs in

- The app sends the email and password to **Supabase**, not to our backend. Our backend never sees a password.
- Supabase checks them and gives the app a **token**: a long piece of text that proves who the teacher is, signed so that it cannot be forged. It is valid for about an hour, and the app renews it automatically in the background.
- From then on, every request the app makes to our backend carries that token. The backend checks the signature, reads the teacher's ID from inside it, and uses that ID in every database query. A teacher can therefore never see another teacher's courses, even by guessing an ID.

What happens when the teacher uploads a file

Photos of class lists, course outlines and course material are uploaded by the app **straight to Supabase Storage**, into a folder named after the teacher's own user ID. The app then sends only the file's path to the backend. Files never pass through the backend on the way in, which keeps uploads fast and keeps the backend simple.

Tools for testing

Every backend answer has the same shape, `{ "success": ..., "data": ..., "message": ... }`, and when a screen shows an error its text comes from `message`.

Tool	What it is for
Swagger at /docs on the backend URL	Every endpoint, with its fields explained. Click Authorize , paste a token, then Try it out and Execute on any endpoint.
Postman	The same requests, saved so they can be repeated. Needed to get a token outside the app (see below).
The seeded demo course	A realistic full semester already filled in: topics, students, a plan with conducted and cancelled classes, attendance, marks. Created by the backend's seed script for a chosen teacher account.
Demo date (<code>EXPO_PUBLIC_DEMO_TODAY</code>)	Makes the app pretend that "today" is a chosen date, so a semester that runs for four months can be shown in one sitting. Leave it empty in a real build.

Getting a token for Swagger or Postman

The app gets its token by itself. To call the backend by hand, sign in to Supabase directly:

- Method **POST**, URL `https://<project>.supabase.co/auth/v1/token?grant_type=password`
- Header `apikey` set to the project's **anon** (publishable) key
- Body, raw JSON: `{ "email": "...", "password": "..." }`

Copy `access_token` from the answer, without the quotes, and paste it into Swagger's **Authorize** box (without the word "Bearer") or into Postman's **Authorization** tab as a **Bearer Token**.

If Supabase answers `unsupported_grant_type`, the `?grant_type=password` part of the URL is missing or has an invisible space or new line after it. Type it by hand rather than pasting it.

The words used in this guide

Word	Meaning
Topic	One item of the syllabus, for example "Normalization". A topic can need several classes.
Slot	A real date on which the course can be taught: a class day that is not a holiday
Session	One planned class: a slot with a topic assigned to it. Its status is planned, conducted or cancelled.
Weightage	How much each kind of assessment counts towards the final total, adding up to 100%
Grade scale	The minimum percentage for each letter grade, set per course
Deficit	How many more classes the remaining topics need than the semester has left
Attendance threshold	The attendance percentage below which a student is flagged (75% unless the teacher changes it)

3. Using the app: signing in and setting up a course

This chapter and the next walk through every page in the order a teacher meets them. Each page lists **what it is for**, **what to do**, and **what to test**, so the same pages double as a test script and as a demonstration script for a teacher.

3.1 Sign in, create account, forgot password

What it is for. Getting into the app. Accounts are handled by Supabase.

What to do.

- **Create account:** full name, email and password, then **Create account**. If the Supabase project asks for email confirmation, the app says **Confirm your email**, and the teacher must click the link in that email before signing in.
- **Sign in** ("Welcome back"): email and password, then **Continue**.
- **Forgot password:** enter the email; Supabase sends a reset link.
- A **Google** sign-in button appears only when it has been switched on for the build.

What to test.

- A wrong password shows "Couldn't sign in", not a crash.
- An empty field shows "Missing details"; a short password shows "Password too short".
- Close and reopen the app: the teacher should still be signed in.
- The name typed at sign-up appears on the profile page and on the reports.

3.2 My courses

What it is for. The list of the teacher's courses. Every other page works inside one chosen course, so this is where the teacher picks it.

What you see. Two tabs, **Active** and **Past**. Each course card shows the course and its state; a course with no timetable yet says **Plan not generated**, and one whose end date has passed says **Semester ended**. A button starts a new course.

What to test. A new teacher sees an empty list with a way to create a course. Tapping a card opens that course on the Dashboard tab. Past courses appear under **Past** once their end date has gone by.

3.3 New course: step 1, course details

What it is for. The only information the planner truly needs about the course.

What to do. Enter the **course name** (for example "Database Systems"), optionally a **course code** (CS-301) and **semester term**, the **start date** and **end date**, and tap the **class days** (for example Mon and Wed). Then **Continue**.

What to test. The end date must be after the start date, and at least one class day must be chosen; otherwise the app shows "Check the details".

3.4 New course: step 2, your topics

What it is for. The syllabus, in teaching order.

What to do. Type or paste the topics into the box, **one per line**, in the order they are taught. Titles only: how many classes each needs is corrected after the plan is seen.

Instead of typing, **Import course outline** lets the teacher photograph the printed outline or choose its PDF. The app reads the topic titles from it and fills the box. If the outline says "Week 3-4: Normalization", that length is applied to the topic automatically.

What to test. Blank lines are ignored. An empty box shows "Check the topics". Import a real outline PDF: the topics should appear in order, and the teacher can still edit the text before continuing.

3.5 New course: step 3, holidays

What it is for. Removing days on which no class can happen.

What you see. Every Pakistani public holiday that falls inside the semester, already switched on. Each shows whether it **Falls on a class day** (it will cost a class) or is **No class that day** (it changes nothing).

What to do. Switch off any holiday the university does not observe, then **Generate my plan**.

What to test. A holiday on a class day really removes that date from the plan. Switching a holiday off puts the date back.

3.6 Plan generated

What it is for. The first result: the whole semester laid out week by week, seconds after setup.

What you see. The **Semester Plan**, grouped by week, with each topic on its dates. A topic that needs several classes appears on consecutive class dates as part 1, part 2 and so on. The page suggests the next step: **Add students to start taking attendance**.

What to test. No class falls on a holiday or on a day that is not a class day. Topics appear in the order they were typed. If the topics need more classes than the semester has, the page says so, and the catch-up options are available from the Plan tab.

3.7 Clone a course (new semester from an old one)

What it is for. Teaching the same course again next term without setting it up from scratch.

What to do. Choose the **source course**, give the **semester name** (for example "Spring 2027") and the new **start and end dates**, then **Clone and generate plan**. The class days are carried over.

What gets copied: the topics with their classes needed, minimums and priorities (each reset to not yet taught), the weightage (how much quizzes, assignments, midterm and final count), and the letter grade scale. Public holidays are worked out again for the new dates. **Students, assessments, attendance and marks stay with the old semester.**

What to test. The new course has the old topics and grading, a fresh plan on the new dates, and no students or assessments.

4. Using the app: running the semester

Inside a course, a bar at the bottom has five tabs: **Dashboard**, **Plan**, **Students**, **Assessment** and **Material**. Attendance, reports, course settings and the profile are opened from these tabs.

4.1 Dashboard tab (the course home)

What it is for. One look that answers "what do I teach today, and am I on track?"

What you see, top to bottom.

- **Today's class**, or the **next class** with its date, and a **Take attendance** button.
- **A warning banner** when the course is behind, in plain words, for example "You are 1 week behind your plan". It links to the catch-up options.
- **Four numbers:** Classes (conducted out of those planned up to today), Progress (percentage of the syllabus taught), Attendance (class average), and Assessments (how many are upcoming).
- **Upcoming this week:** the other classes in the next seven days.

What to test. Mark a class conducted and see **Classes** and **Progress** rise. Leave a past class unmarked and see the behind-schedule warning appear. Pull down to refresh.

4.2 Plan tab: the teaching plan

What it is for. The whole timetable, and the place where the teacher records what really happened.

What you see. The **Teaching Plan** in two views: **Calendar** (class days highlighted, with cancelled classes and quizzes marked) and **Week list**. Each class shows its topic, and conducted classes show **Session recorded**.

What to do. Tap a class to open its details. The menu offers **Edit topics**, **Catch-up options** and **Rebuild plan from today**.

4.3 A class's details: conducted or cancelled

This sheet is where most of the planner's work is triggered.

- **Take attendance:** opens the attendance page for this class. Everyone starts as present. Submitting attendance also marks the class conducted.
- **Mark conducted:** records that the class happened **without taking the roll**.
- **Mark cancelled:** asks **Why is this class cancelled?** with quick choices (Public holiday, Campus closed, I was unavailable, Exam duty) or a typed reason. **Cancel class and replan** rebuilds the rest of the semester and shows what moved. The reason is printed on the course report. **Keep this class** backs out.
- **Attach material:** upload slides or notes for this class's topic.

What to test. Cancel a future class: its topic moves to the next free class date, and every later topic shifts with it. Cancel a class and then check that conducted classes before it did not change.

4.4 Plan updated ("what moved")

What it is for. Showing the teacher, in plain sentences, what the replan changed, instead of silently changing the timetable.

What you see.

- **Past sessions (locked):** conducted classes, which a replan never touches.
- **Schedule shifts:** each topic that moved, from which date to which date. If nothing moved, it says **Nothing had to move**.
- **Assessment rescheduled:** any quiz that had to move because its topics now finish later, with the reason, for example "Quiz 3 moved from 18 Sep to 25 Sep because Normalization will not be completed before 24 Sep."
- **No longer fits:** topics that no longer fit in the semester, with **See ways to catch up**.

4.5 Edit topics

What it is for. Correcting the defaults after seeing the plan.

What you see. The topics in teaching order with their status (**In progress, Taught, Dropped**), and totals: **Classes left**, what the **Topics need**, and what they need **At minimum**. Tapping a topic opens a sheet with:

- **Sessions needed:** the classes this topic normally takes.
- **Minimum sessions:** the fewest it could survive on (used by the Compress option).
- **Priority:** low, normal or high. Low-priority topics are the first candidates for dropping.

Rebuild plan applies the changes. **What to test:** raise a topic's sessions needed and see later topics shift and, if needed, a quiz move.

4.6 Catch-up options (the deficit screen)

What it is for. The moment the remaining topics need more classes than the semester has left. The app does not show an error. It works out three ways to recover and lets the teacher choose.

What you see. **Semester pacing deficit:** how many classes short the course is, and a bar of **Available vs needed**. Then three **Recovery strategy** cards, each saying how many classes it recovers and whether that is enough:

- **Drop topics:** remove whole topics, choosing the best combination (never a more important topic when less important ones are enough).
- **Compress topics:** teach some topics in fewer classes, never below their minimum.
- **Add makeup classes:** extra classes on days the course is not normally taught (not Sundays or holidays), with the suggested dates.

The app pre-selects the first option that fully solves the problem, checking Compress, then makeup classes, then Drop. The teacher taps **Choose**, the option is applied, the plan is rebuilt, and the "what moved" page follows. Classes already taught are never changed.

What to test. Set topics to need more classes than exist (for example, raise several topics' sessions needed), open the options, and check that each card's numbers add up. Apply each option on a copy of a course and check the new plan.

This page is the best part of the app to demonstrate. How each option is calculated is explained in chapter 5.

4.7 Students tab: adding the class list

What it is for. Getting every enrolled student into the course **without typing them**.

While the course has no students, the **Student Roster** page offers three ways in:

- **Capture Class List:** opens the camera. Photograph the printed class list; for several pages, take one photo per page. A photo takes up to about thirty seconds to read. Straight, flat and well-lit pages read best.
- **Import file:** choose the PDF the department sent (or an image). A department PDF is read **exactly**, in about a second.
- **Paste list:** the fallback. One student per line, roll number then full name, separated by a comma or a tab.

Whichever way is used, nothing is saved yet. The rows go to the review page.

4.8 Review class list (mandatory)

What it is for. A misread roll number would silently attach one student's attendance and marks to the wrong person for the whole semester. So the rows read from a photo or file are only a **draft**, and the teacher confirms them.

What you see. An editable table of **Roll no** and **Full name**. Rows are marked when:

- the roll number is missing, or the name is missing;
- the same roll number appears twice (flagged, never merged: merging would lose a student);
- the reading is doubtful, for example the photo was blurred there (highlighted as a warning to double-check).

Missing values and duplicates must be fixed before saving. A doubtful row can be saved once the teacher has looked at it.

What to do. Correct any cell, delete wrong rows, **Add a row** by hand for anyone missed, and **Capture another page** to append the next page of the list. Then **Review and add** saves the students.

What to test. Photograph a real class list: seat numbers (EB followed by 11 digits on the Computer Science sheet) should match the paper. Type a duplicate roll number and check that saving is blocked. Import the same list twice: existing students are matched by roll number, not added again.

4.9 Student list and student profile

What it is for. Seeing who is enrolled and who is at risk.

What you see. All enrolled students with their attendance percentage; students below the attendance threshold are highlighted. A search box finds a student by name or roll number.

Tapping a student opens the **Student Profile**: their attendance class by class, and their marks in every assessment. **Remove from this course** takes the student off this course's list; their record is kept.

4.10 Attendance

What it is for. Taking the roll in a few taps, and knowing who is below the threshold.

How to open it. **Take attendance** on the Dashboard tab, or on a class in the Plan tab. If several classes are waiting, the page first asks the teacher to **Choose a class**.

What to do. Every student starts as **Present**. Tap a student's row to mark them **Absent**; tap again to undo. Then **Submit attendance**. Submitting also records the class as conducted. Opening a class that already has attendance shows "Already recorded — submitting again replaces it", so mistakes can be corrected.

See summary shows each student's percentage, the class average, and an **Action required / Exam eligibility warning** list of students below the threshold. Before any class has been conducted, it explains that there is nothing to average yet. **Download signed PDF roster** produces the attendance report.

What to test. Mark two students absent in one class and check both their percentages. Edit that attendance and check the percentages update. Check that the threshold list matches the threshold set in course settings.

4.11 Assessment tab

What it is for. Every quiz, assignment, midterm, final and participation mark, and the grading that turns them into a result.

What you see. The assessments with their status: **scheduled** (a date is set), **unscheduled** (no date yet), **needs marks** (the date has passed and marks are missing), and **graded** (every student has a mark). A **Grading Criteria** card shows the weightage, and **Results & grades** opens the results.

Create an assessment

Choose the **type**, a **date** (or **Set later**), the **total marks**, and the **topics it covers**. If the chosen date falls before those topics are taught, a suggestion appears at once, for example "These topics are taught until 24 Sep. Move to 25 Sep?", with a **Move** button. The backend enforces the same rule on every replan: the planner never leaves an assessment before its topics.

Enter marks

One student at a time, with a **number keypad**. Type the score and the page moves to the **Next student** automatically. Each student can be:

- **Mark entered:** a score.
- **Marked absent:** counts as 0. The student really did not take it.
- **Not entered yet (Leave blank):** no mark yet. This is **not** treated as zero; the result is shown as provisional until it is entered.

Discard leaves without saving; saving asks for confirmation.

Grading criteria

- **Component weightage:** how much quizzes, assignments, midterm, final and participation count. It must total exactly 100%.
- **Grade bands:** the minimum percentage for each letter grade. A student gets the highest grade whose minimum they reach. It starts from a standard scale (A from 85%, A- from 80%, B+ from 75% and so on) that the teacher can change or extend with + **Add grade**, because grading differs between universities.

Results & grades

Each student's weighted total and letter grade, with the **Class Average** and **Highest** at the top and a search box. A student with any mark still missing is labelled **Provisional**. **Result sheet (PDF)** produces the result document. These numbers come from the same calculation as the PDF, so the screen and the document can never disagree.

What to test. Set weightage that does not total 100 and check it is refused. Enter marks, leave one blank, and check that student shows as provisional rather than as a low grade. Mark one absent and check it counts as zero.

4.12 Material tab

What it is for. Keeping slides, notes, assignments and papers with the course, organised by topic.

What you see. Folders, one per topic, and filters such as **All Files** and **Slides**, with a search box.

What to do. Upload file: pick the file (PDF, PPTX, DOCX, ZIP, up to 50 MB), give it a title, and choose the **topic**, which sets the folder. Material can also be attached from a class in the Plan tab.

4.13 Reports

What it is for. The three documents a teacher is asked for, produced from data the app already has.

Report	What it contains
Result Sheet	Course, code, semester, teacher and department; every student's roll number, name, total per component, weighted total and grade; class summary (number of students, average, highest, lowest, grade distribution, pass count); generation date and a signature line
Attendance Report	Class-by-class attendance for every student, their percentages, and the students below the threshold
Course Delivery Report	Every class, what it covered, and why any class was cancelled

What to do. Open **Reports** from the **More & Settings** page, preview a report, and share or save it as a PDF.

What to test. The result sheet's totals match the **Results & grades** page exactly. The course delivery report shows the cancel reasons that were typed on the Plan tab.

4.14 Course settings

- **Edit course info** (name, code, semester, dates) and **Class Days**. Changing the dates or class days rebuilds the plan, adding any public holidays the new dates cover.
- **Holidays & Off Days**: each public holiday switched on or off. To skip one ordinary class day, cancel that class from the Plan tab instead.
- **Attendance threshold**: a slider, 75% by default. **Save Changes** saves everything.
- The **More & Settings** page (the profile) holds the teacher's name and email, and shortcuts to **Course Material**, **Reports**, **Switch course**, **Clone Semester** and **Sign Out**.

5. The algorithms: what they are, where they run, and why

UstaadAssist is a project for a Design and Analysis of Algorithms course, and the planner is where that shows. Every part of it is a **pure function**: it takes data in and returns data out, and never reads or writes the database itself. The backend loads the data, calls the function, and saves the result. That is what makes each algorithm testable on its own, and every one below has automated tests.

In this chapter, **n** means the number of topics, **S** the number of class slots in the semester, **D** the number of days between the start and end dates, and **A** the number of assessments.

5.1 Summary

File (in backend src/)	Algorithm	What it does for the teacher	Time
<code>planner/slots.ts</code>	Linear calendar walk with hash-set lookup	Finds every real class date, with holidays removed	$O(D)$
<code>planner/allocate.ts</code>	Single-pointer sequential allocation	Lays the topics across those dates in order	$O(n \log n + S)$
<code>planner/assessments.ts</code>	Hash map of finish dates + binary search	Keeps every quiz after the topics it covers	$O(S + A \log S)$, plus the topics each quiz covers
<code>planner/replan.ts</code>	Freeze-and-rebuild replanning, with a per-topic diff	Rebuilds the future after a cancellation and lists what moved	$O(S + n \log n)$
<code>planner/deficit.ts</code> (Drop)	0/1 knapsack by dynamic programming	Chooses the best set of topics to drop	$O(n \times T)$, T = classes the topics need
<code>planner/deficit.ts</code> (Compress)	Greedy by largest slack (provably optimal here)	Shortens topics without going below their minimum	$O(n \log n)$
<code>planner/slots.ts</code> (Extend)	Greedy earliest free dates	Suggests makeup class dates	$O(D)$
<code>planner/health.ts</code>	Counting formula	"You are 1 week behind your plan"	$O(S)$
<code>extraction/classList.ts</code>	Majority voting across several OCR readings	Reads a class list photo reliably	linear in the text read
<code>grading/compute.ts</code>	Weighted sum, rounded once	Totals and letter grades	$O(\text{students} \times \text{marks})$

5.2 Slot generation: which dates can have a class

The planner walks every date from the first day of the semester to the last. A date is kept when its weekday is one of the class days and it is not a holiday.

The holidays are first put into a **hash set** (a JavaScript `Set`). Checking "is this date a holiday?" in a set takes constant time, $O(1)$, however many holidays there are, instead of scanning the holiday list for every day. The walk is therefore $O(D)$: one step per calendar day. Each kept date also gets a week number (week 1 is the first

seven days, and so on).

5.3 Topic allocation: which topic on which date

The topics are sorted by their teaching order ($O(n \log n)$). Then one pointer moves forward through the list of slots: each topic takes the next **sessions needed** slots, and the pointer moves on. A topic needing three classes gets three consecutive class dates, numbered part 1, 2 and 3.

A topic is never split: if not enough slots remain for the whole topic, it goes into an **overflow** list, which is what the deficit screen later deals with. Each slot is visited at most once, so after sorting the work is $O(S)$.

5.4 Assessment validation: a quiz never comes before its topics

- First, one pass over the plan records the **last date each topic is taught**, in a hash map (topic to date). $O(S)$.
- For each assessment, the latest of those dates among the topics it covers is the date the assessment must come after.
- If the assessment is earlier, it must move to **the first class date after that day**.

That last step is now a **binary search**. The slots are already in date order, so there is no need to check them one by one from the start. Look at the middle slot: if it is on or before the date, the answer can only be to its right; otherwise it may be the answer, so keep it and look to its left. Each step throws away half of what is left, so a semester of S slots takes about $\log_2(S)$ steps instead of up to S .

```
firstSlotAfter(slots, date):
    low = 0, high = number of slots
    while low < high:
        middle = (low + high) / 2, rounded down
        if slots[middle].date <= date: low = middle + 1
        else: high = middle
    return slots[low] if low is inside the list, otherwise "none"
```

A reason is written for the teacher every time something moves, for example "Quiz 3 moved from 18 Sep to 25 Sep because Normalization will not be completed before 24 Sep."

Why binary search here, when a semester only has about 40 class dates? The speed-up is small at this size. The point is that the list is **already sorted**, and a sorted list should be searched by halving. The tests check that it always returns exactly what a plain left-to-right search returns, for every date across a whole semester.

5.5 Replanning: freeze the past, rebuild the future

When a class is cancelled, or a topic's length changes, the planner does not start from scratch:

- Classes already **conducted** are never touched.
- Only the teaching still owed is worked out: for each topic, its sessions needed minus the classes already given to it.
- Slots are generated again **from today onward**, keeping the course's own week numbers, and the remaining topics are allocated into them (5.3).
- Assessment validation (5.4) runs again.

Then the planner **compares the old plan with the new one**, topic by topic: the old dates of a topic are paired with its new dates in order, and each pair that differs becomes one line on the "what moved" page. A cancelled class is paired with the class that replaces it, so the teacher reads "Joins moved from 14 Sep to 21 Sep" rather than a mysterious new class.

5.6 Deficit resolution: three ways to recover

When the topics still to teach need more classes than the semester has left:

```
deficit = classes the remaining topics need - class slots still available
```

If the deficit is above zero, the planner does not report an error. It calculates three options.

Drop: the 0/1 knapsack problem, solved by dynamic programming

The problem. Choose some remaining topics to drop so that together they free at least **deficit** classes, while losing as little as possible. Each topic is either dropped completely or kept completely ("0/1"), and each frees a known number of classes (its "weight"). This is the classic **0/1 knapsack** problem.

What counts as losing as little as possible, in this order:

- never drop a high-priority topic if lower-priority topics can cover the deficit, and the same for normal priority;
- then drop as few topics as possible;
- then free as few extra classes as possible (the closest to exactly the deficit);
- then prefer topics later in the course, because earlier topics tend to be foundations.

Why the obvious approach fails. The simple, **greedy** approach takes the least important topic, then the next, until the deficit is covered, and never reconsiders. Suppose the course is 3 classes short, with three low-priority topics:

Topic	Classes it needs
Topic 3	3
Topic 4	2
Topic 5	2

Greedy takes Topic 5 (2 classes), then Topic 4 (2 more): **two topics dropped, and 4 classes freed when 3 were needed**. Dropping Topic 3 alone frees exactly 3 and loses only one topic. Greedy cannot find that, because it committed to Topic 5 at the first step. The app used to behave exactly like this; it now uses dynamic programming.

Dynamic programming means solving small versions of the problem first, storing their answers in a table, and building the full answer from the stored ones, instead of committing to one choice at a time. The table here is:

```
best[i][s] = the cheapest way to free EXACTLY s classes
             using only the first i topics    (or "impossible")
```

Every cell has just two choices for topic i: skip it, or drop it on top of the best way to free the other s - weight classes:

```
best[i][s] = the better of  best[i-1][s]                (skip topic i)
                           best[i-1][s - weight(i)] + topic i (drop topic i)
```

The first row is simple: with no topics, only "free 0 classes" is possible, and it costs nothing. The table is filled row by row. The answer is the cheapest cell in the last row whose s is at least the deficit. A second table remembers, for each cell, whether topic i was dropped, so the chosen topics can be read back by walking up the table from the answer.

For the example above, the last row finds that $s = 3$ is reachable by dropping only Topic 3 (one topic), while $s = 4$ needs two topics, so it chooses Topic 3.

Cost. The table has $(n + 1)$ rows and $(T + 1)$ columns, where T is the total number of classes the remaining topics need, and each cell takes constant time: **$O(n \times T)$** time and memory. Trying every combination of topics instead would take $O(2^n)$: about a million combinations for 20 topics, compared with a few thousand table cells.

How we know it is right. Besides tests for the example above, for priorities, and for the tie-breaks, one test builds 500 random small courses, finds the best answer by trying every possible combination of topics, and checks that the dynamic programming finds an equally good one every time.

Compress: greedy, and here greedy is provably optimal

Compress shortens topics that currently take more classes than their minimum. Each topic's **slack** is its sessions needed minus its minimum sessions. The planner sorts topics by slack, largest first, and takes classes from each until the deficit is covered, never going below a minimum.

Greedy **is** optimal here, unlike in Drop. Every class removed is worth the same, one class, and a topic can give up any number of classes up to its slack. So taking from the topic with the most slack first always recovers the most classes while touching the fewest topics, and no other choice can do better. This contrast is worth knowing: greedy works when choices can be taken **partially** and are all worth the same (like the fractional knapsack), and fails when each choice is **all or nothing** (the 0/1 knapsack).

Extend: makeup classes

The planner walks forward from today and collects the first **deficit** dates that are not normal class days, not Sundays, and not holidays. The earliest dates are chosen because a makeup class is most useful before the topics that need it. This is $O(D)$ in the worst case.

The teacher picks one option. It is applied (topics dropped, topics shortened, or makeup dates added as class slots), and the planner runs again.

5.7 Schedule health

```
behind_by_weeks = (classes planned up to today - classes conducted) / classes per week
```

Cancelled classes are left out of "planned up to today", because a cancelled class was never an opportunity to teach. The result, rounded to one decimal place, becomes the warning on the Dashboard tab.

5.8 Reading a class list photo: voting between readings

A single OCR reading of a phone photo makes mistakes. So the backend reads the photo six ways (the Tesseract OCR engine in three page-layout modes, each with and without automatic rotation), parses the roll numbers and names from each reading, and lets the readings **vote**:

- Rows are matched between readings by roll number (and by how many times that number appeared, so a roll number genuinely printed twice stays two rows).
- For each row, the spelling of the name that most readings agree on wins. OCR often drops a space between words but almost never adds one, so spellings that differ only by spaces count as the same vote.
- A row only a few readings saw, or on whose name the readings disagree, gets a **low confidence**, and the review page highlights it.

A department PDF is read from its text layer instead, which is exact. Either way, the teacher's review is the final word (4.8).

5.9 Results: one calculation for the screen and the PDF

For every student: the marks in each component (quizzes, assignments, midterm, final, participation) are summed and turned into a percentage of that component's total, then combined using the teacher's weightage, and the letter grade is the highest band whose minimum the student reaches. Two rules keep it honest:

- **Round once, at the end.** Rounding each component first would add up small errors.
- **A missing mark is not a zero.** It is reported as missing and the result is shown as provisional; only a student marked absent scores a real zero.

The Results & grades page and the result sheet PDF call this same function, so they can never disagree.

6. Showing it to a teacher, and testing it

6.1 A ten-minute demonstration

This order tells the story the way a teacher lives it: set up in two minutes, then watch the app keep the plan correct when the semester goes wrong. Use a fresh course for steps 1 to 4, and the seeded demo course (with the demo date set) for steps 5 to 9.

- **1. The problem (1 minute).** Ask the teacher how they plan a semester and what happens to the plan when classes are lost. Most will describe a written outline that stops matching reality within weeks.
- **2. Setup (2 minutes).** Create a course: name, dates, two class days. Paste ten topic titles, one per line. Show the holidays already filled in. Tap **Generate my plan**.
- **3. The plan.** Show the week-by-week plan. Point out that no class falls on a holiday, and that a topic needing two classes takes two consecutive class dates.
- **4. The class list.** Photograph a printed class list, or import the department PDF. Show the review page, and a highlighted row the teacher has to check.
- **5. Home.** Open the seeded course. Show today's class, the four numbers, and the "behind your plan" warning.
- **6. A cancelled class.** On the Plan tab, cancel a class with the reason "Campus closed". Show the "what moved" page: past classes locked, topics shifted, and a quiz moved with its written reason.
- **7. Running out of time.** Raise a topic's classes needed until the course no longer fits. Open the catch-up options and explain the three cards. Choose one and show the rebuilt plan.
- **8. Attendance and marks.** Take attendance by tapping two absentees. Enter marks for a quiz with the keypad, leaving one blank, and show that student as provisional on **Results & grades**.
- **9. Reports.** Open the result sheet and the course delivery report, which lists the cancelled class and its reason.

The line that lands with teachers: "You type your topic titles once. When a class is cancelled, you tap one button and the whole semester, including your quiz dates, is fixed for you."

6.2 Test checklist

Each row is one thing to confirm on a real phone, against the real backend.

Page	Check
Sign in	Wrong password shows an error; the session survives closing the app
Create account	The full name appears on the profile and on reports
My courses	Active and Past tabs split courses by end date; a card opens its course
New course	Invalid dates or no class days are refused; blank topic lines are ignored
Outline import	A real outline PDF fills the topics in order; "Week 3-4" gives that topic two weeks of classes
Holidays	A holiday on a class day removes that date; switching it off restores it
Plan generated	No class on a holiday or a non-class day; multi-class topics are consecutive

Page	Check
Clone	Topics, weightage and grade scale copied; no students or assessments; new dates planned
Dashboard	Numbers change after marking a class conducted; the behind warning appears when classes are unmarked
Plan: cancel	Future topics shift; conducted classes do not change; the reason is saved
What moved	Every shift listed; a moved quiz shows its reason
Edit topics	Changing classes needed or priority rebuilds the plan
Catch-up options	Each card's numbers add up; Drop never cuts a higher-priority topic when lower ones suffice; applying an option rebuilds the plan
Class list photo	Seat numbers match the paper; doubtful rows highlighted
Review page	Missing values and duplicates block saving; add row, delete row and extra pages work
Students	Attendance percentages shown; below-threshold students highlighted; search works
Student profile	Attendance and marks shown; removing a student keeps their record
Attendance	Everyone starts present; editing replaces the old record; the summary matches
Create assessment	A date before its topics triggers the Move suggestion
Enter marks	The keypad moves to the next student; blank stays blank; absent counts as zero
Grading criteria	Weightage must total 100; a new grade band takes effect
Results & grades	Provisional shown for missing marks; the average matches the result sheet
Material	Upload, find in its topic folder, open, delete
Reports	All three generate; the result sheet matches the results screen
Course settings	Changing dates or class days offers to rebuild the plan; the threshold changes the flagged list
Sign out	Asks for confirmation; returns to sign in

6.3 Worth knowing while testing

- **A photographed class list can take up to about thirty seconds** to read, because it is read six times and the readings vote. The department's PDF takes about a second.
- **The demo date.** With `EXPO_PUBLIC_DEMO_TODAY` set, the app and the backend both treat that date as today. Clear it before building the app for real use, or every day will look like that date.
- **Tokens expire after about an hour.** Inside the app this is invisible. In Swagger or Postman, a sudden 401 means it is time to sign in again and paste a new token.
- **When a screen shows something unexpected,** repeat the request in Swagger with the same account. A right answer there means the problem is in the app; a wrong one means the backend.